Arduino for Embedded Systems - Beginner Level

Lesson 3: Digital I/O: LEDs and Buttons

Digital I/O: LEDs and Buttons – Study Notes

Key Concepts

1. **Pin Configuration** – Setting a microcontroller pin as `INPUT`, `OUTPUT`, or `INPUT_PULLUP`.
2. **Reading Button State** – Using `digitalRead()` (or equivalent) to detect HIGH/LOW logic levels.
3. **Controlling an LED** – Driving an LED with `digitalWrite()` and proper current limiting resistors.
4. **Debounce Logic** – Eliminating false triggers caused by mechanical bounce, via software timing or hardware RC filtering.
5. **Pull up / Pull down Resistors** – Internal vs. external resistors to define a known default state for a button.

Summary

In this lesson you learned how to interface simple digital peripherals—LEDs and push buttons—with a microcontroller (e.g., Arduino, ESP32, STM32). The first step is to configure the I/O pins: the LED pin must be an **output**, while the button pin is an **input** (often with an internal pull up enabled). When the button is pressed, its voltage level changes, and `digitalRead()` reports the new state.

Because mechanical switches do not transition cleanly, they generate rapid on/off “bounces” that can be misinterpreted as multiple presses. Debounce techniques—either a short delay after a state change, a time stamp comparison, or a hardware RC filter—ensure that only a single, clean transition is acted upon. With a reliable button reading, you can safely toggle or dim an LED, build simple user interfaces, or chain more complex logic.

Important Points

• **Pin Modes**

- `pinMode(pin, OUTPUT);` – drives an LED or other actuator.
- `pinMode(pin, INPUT);` – reads a raw button signal (requires external resistor).
- `pinMode(pin, INPUT_PULLUP);` – enables the MCU’s internal pull up resistor (common for buttons).

• **Wiring the LED**

- Connect the LED’s **anode** to the output pin through a current limiting resistor (typically 220 /:™c33 ó© for 5 /V).
- Connect the **cathode** to ground.

• **Wiring the Button**

- **Active LOW** (most common with `INPUT_PULLUP`): one side to the input pin, the other side to ground.
- **Active HIGH** (using external pull down): one side to the input pin, the other side to VCC, plus a pull down resistor (10 /k:’’à

• **Debounce Strategies**

1. **Software delay** – `delay(10);` after detecting a change (simple but blocks other code).
2. **Time stamp method** – store `millis()` when the state first changes; ignore further changes until a set interval has passed.
3. **State machine method** – track `stableState`, `lastReading`, and `lastDebounceTime`.
4. **Hardware RC filter** – a ~10 /k:' &W6—7F÷" ² ã ûTb 6 6—F÷" 7&V FW2 ã ö×2 Æðw pass filter.

• **Common Pitfalls**

- Forgetting the current limiting resistor !' LED burns out.
- Wiring the button without a defined default !' floating input !' random readings.
- Using `delay()` for debounce in time critical projects; prefer non blocking methods.

Examples

Example 1 – Simple LED Toggle with Button (Arduino style)

```cpp
const uint8_t LED_PIN    = 13;   // built in LED on many boards
const uint8_t BUTTON_PIN = 2;    // push button connected to D2
bool ledState = false;          // current LED state
bool lastButtonState = HIGH;    // because we use INPUT_PULLUP
unsigned long lastDebounce = 0;
const unsigned long DEBOUNCE_MS = 50;   // 50 ms debounce interval
void setup() {
  pinMode(LED_PIN, OUTPUT);
  pinMode(BUTTON_PIN, INPUT_PULLUP);   // enable internal pull up
  digitalWrite(LED_PIN, LOW);
}
void loop() {
  bool reading = digitalRead(BUTTON_PIN);
  // If the reading changed, reset the debounce timer
  if (reading != lastButtonState) {
    lastDebounce = millis();
  }
  // Only act if the reading has been stable for DEBOUNCE_MS
  if ((millis() - lastDebounce) > DEBOUNCE_MS) {
    // When the button is pressed (LOW) and we have a stable change
    if (reading == LOW && lastButtonState == HIGH) {
      ledState = !ledState;              // toggle LED state
      digitalWrite(LED_PIN, ledState ? HIGH : LOW);
    }
  }
  lastButtonState = reading;   // store for next loop
}
```

```

**Explanation**

- The button is wired **active LOW** using the internal pull up.
- A classic debounce state machine checks that the reading stays unchanged for 50 /ms before accepting it.
- Each valid press toggles the LED.

---

### Example 2 – Fade an LED with Button Press (Non blocking debounce)

```cpp
const uint8_t LED_PIN    = 9;      // PWM capable pin
const uint8_t BUTTON_PIN = 4;
int brightness = 0;               // 0 255 PWM value
int fadeAmount = 5;               // step size
bool buttonPressed = false;
unsigned long lastDebounce = 0;
const unsigned long DEBOUNCE_MS = 30;
void setup() {
  pinMode(LED_PIN, OUTPUT);
  pinMode(BUTTON_PIN, INPUT_PULLUP);
}
void loop() {
  // ----- Debounce -----
  bool reading = digitalRead(BUTTON_PIN);
  if (reading == LOW && (millis() - lastDebounce) > DEBOUNCE_MS) {
    buttonPressed = !buttonPressed;   // toggle mode on each press
    lastDebounce = millis();
  }
  // ----- LED control -----
  if (buttonPressed) {
    // Fade up/down continuously
    analogWrite(LED_PIN, brightness);
    brightness += fadeAmount;
    if (brightness <= 0 || brightness >= 255) {
      fadeAmount = -fadeAmount;   // reverse direction
    }
    delay(10);                    // small delay for visible fade
  } else {
    analogWrite(LED_PIN, 0);      // LED off when not in fade mode
  }
}
```
```

Explanation

- Pressing the button toggles **fade mode** on/off.
- The debounce uses a short, non blocking check (no long `delay()` inside the debounce block).
- When active, the LED brightness is varied with PWM (`analogWrite`).

Quick Review

1. **What pin mode should you use for a button when you rely on the MCU's internal pull up resistor?**
2. **Why is a current limiting resistor required for an LED, and how do you choose its value?**
3. **Describe one software debounce technique and its advantage over a simple `delay()`.**
4. **What is the default logic level read from a button wired to `INPUT_PULLUP` when the button is not pressed?**
5. **How would you modify Example /1 to make the LED turn **off** instead of toggle on each press?**

Further Reading

- **Interrupt Driven Button Handling** – using `attachInterrupt()` for edge triggered, low latency response.
- **Hardware Debounce Circuits** – RC networks, Schmitt triggers, and dedicated debounce ICs (e.g., MAX6816).
- **PWM & LED Brightness Control** – deeper dive into duty cycle, frequency considerations, and LED dimming techniques.
- **Power Efficient I/O** – configuring pins for low power sleep modes, using `pinMode(INPUT_PULLDOWN)` where available.
- **Multiple Button Scanning** – matrix keyboards, multiplexing, and debouncing many switches efficiently.